

Explicando Decisões de LLMs via Otimização Inversa

Dissertação de Mestrado — Execução produção 30/06–01/07/2026 (3 blocos, 3 seeds)

George Gomes da Silva

Programa de Pós-Graduação em Informática
Orientador: Prof. Raul Fonseca Neto (UFJF)

Julho de 2026

Pergunta central: LLMs possuem um processo decisório estável e aprendível?

- LLMs são caixas-pretas: bilhões de parâmetros, decisões em texto livre.
- Pedir explicação em linguagem natural dá *racionalização*, não causalidade.
- Aplicações críticas (saúde, crédito, jurídico) exigem explicação verificável.

Proposta: em vez de pedir explicação, *observar* decisões e *recuperar matematicamente* o critério subjacente via **otimização inversa**.

- H1 — **Consistência (Bloco 1)** $\hat{W}_{\text{LLM}}$ estimada em A prevê classificações em B e C ($\text{Kappa} > 0,5$).
- H2 — **Few-shot amplifica (Bloco 1)** Exemplos rotulados aumentam concordância LLM–métrica.
- H3 — **LLM como aprendiz (Bloco 2)** Via in-context learning, o LLM reproduz o critério de um perito externo.
- H4 — **Exemplos difíceis (Bloco 2)** *Hard* ensina mais que *easy*. Refutada em n_{shot} pequeno; revertida em $n_{\text{shot}} = 40$.
- H5 — **Estabilidade (transversal)** Resultados reproduzíveis entre sementes.

$$\hat{y}(x) = \arg \min_k d_W(x, c_k), \quad d_W(x, c) = \sum_{j=1}^d w_j (x_j - c_j)^2$$

Problema inverso: dados pares $(x_i, \hat{y}_i)$ do LLM, encontrar $W \geq 0$ tal que $\hat{y}_W(x_i) = \hat{y}_i$ para o máximo de pontos.

Dois algoritmos independentes:

- **Perceptron Estruturado** (Coelho, Borges & Fonseca Neto, CILAMCE 2017): relaxação de margem + busca binária em γ .
- **NNLS** (Lawson & Hanson, 1974): $\min \|Aw - b\|^2$ s.t. $w \geq 0$, sem hiperparâmetros.

Distância: $d_W(x, c) = \sum_{j=1}^d w_j(x_j - c_j)^2$, $w_j \geq 0$. **Classificador:** $\hat{y}(x) = \arg \min_k d_W(x, c_k)$.

Conexão Mahalanobis cheia: $W = \Sigma^{-1}$ diagonal $\Leftrightarrow$ features independentes (naive Bayes na métrica).

Margem geométrica: $\gamma(x) = d_W(x, c_{\text{errado}}) - d_W(x, c_{\text{certo}})$.

Diagonal: $O(d)$ parâmetros vs. $O(d^2)$ na cheia; identificável com $n \ll d^2$; interpretabilidade direta.

Nomenclatura: **A** (treino), **B/C** (transferência, $\pm 1,5$), **D** (meia-lua não-linear), **E** (LLM aprendiz).

Parâmetro	Valor (produção)
Modelo LLM	openai/gpt-4o-mini
Temperatura	0,0
Sementes	[42, 123, 7]
Repetições	3 por configuração (9 observações/célula)
Few-shot sizes (Problema E)	[0, 5, 10, 20, 40]
Estratégias	easy, hard, mixed, random
Experts (Problema E)	aniso_x2, aniso_x1, euclidean
Hiperparam. Perceptron	$\eta = 0,001$, $C = 1,0$ (CILAMCE 2017)
Concorrência	Semaphore(10) + timeout 60s
Chamadas ao LLM	119 713 (fallback do parser = 0,00%)
Flip $T=0$ (auditoria offline)	4,753% (1 384 de 29 116 queries repetidas)
Coleta	30/06–01/07/2026, alias gpt-4o-mini (sem <i>snapshot</i> datado)
Início execução	30/06/2026 23:46

Algoritmo 1 — Perceptron Estruturado (pseudocódigo)

Referência: **Coelho, Borges & Fonseca Neto (CILAMCE 2017, Seção 5.2).**

```
ENTRADA: X (m x d), y (m,), centroids (K x d)
         eta=0.001, C=1.0, delta_gamma=0.05, max_epochs=100, tol=1e-5
SAIDA:   w (d,), gamma_otimo

Inicializacao: w<-ones(d); alpha<-zeros(m); gamma_lo<-0; gamma_hi<-inf; lambda<-1/C

# Busca binaria em gamma (loop externo, Eq. 31):
repita ate |gamma_hi - gamma_lo| <= tol:
    gamma <- gamma_init (+ delta_gamma a cada iter viavel)

    # Perceptron interno (loop sobre epocas):
    para cada ponto i em ordem aleatoria:
        k <- argmin distancia para centroide RIVAL
        l <- classe verdadeira (y[i])
        margem <- d_W(x_i, c_k) - d_W(x_i, c_l) + lambda * alpha[i]
        se margem < gamma * ||w||:           # violacao Eq. 28
            g <- (x_i - c_l)^2 - (x_i - c_k)^2
            w <- max(0, w * (1 - eta*gamma/||w||) - eta*g)   # Eq. 29, projecao SDP
            alpha[i] += eta

    se zero_violacoes: gamma_lo<-gamma; w_star<-w; gamma<-gamma+delta_gamma
    senao:               gamma_hi<-gamma; gamma<-(gamma_lo+gamma_hi)/2

retorna w_star, gamma_lo
```

Hiperparâmetros do artigo (Seção 6, p. 16): $\eta = 0,001$, $C \in [0,1; 1,0]$.

Diagnóstico: `gamma_history` capturada em cada iteração (figura `final_10_gamma_convergence.png` no Apêndice).

Algoritmo 2 — NNLS (pseudocódigo)

Referência: **Lawson & Hanson (1974)**, método de conjunto ativo.

ENTRADA: X ($m \times d$), y (m), centroids ($K \times d$)

SAÍDA: w (d ,) com $w \geq 0$

```
# Construção da matriz A (m x d) e do vetor b (m,):
para cada ponto i:
  l <- classe verdadeira (y[i])
  k <- centroide rival (k != l)
  para cada dimensão j:
    A[i, j] <- (x[i, j] - c[k, j])^2 - (x[i, j] - c[l, j])^2
    b[i] <- 1                # margem-alvo; razão w invariante a escala

# Resolver via scipy:
w <- scipy.optimize.nnls(A, b)

# Formulação matemática equivalente:
min ||A w - b||^2      sujeito a   w >= 0

# Normalização opcional:
w <- w / ||w||

retorna w, gamma=0      # NNLS não define margem geométrica explícita
```

Vantagens sobre Perceptron Estruturado:

- Zero hiperparâmetros (determinístico, reproduzível bit-a-bit).
- Otimalidade global garantida (problema QP convexo).
- Solução única em problemas bem-condicionados ($\text{rank}(A) = d$).
- Cosseno típico com W do Perceptron: $> 0,97$ — validação cruzada do método.

Pipeline com 5 camadas de proteção:

- 1 Parser de 7 camadas para respostas malformadas.
- 2 `MAX_FORMAT_RETRIES` = 5 reenvios com reforço da instrução.
- 3 Fallback determinístico por hash MD5 das coordenadas (sem viés geométrico).
- 4 Concorrência assíncrona: `asyncio.Semaphore(10)` — $\sim 10\times$ speedup.
- 5 Timeout 60s + `asyncio.wait_for(90s)` contra travamento.

Bloco 1 — LLM como FONTE

Pergunta central: o LLM tem um critério decisório estável e aprendível?

- **Setup:** LLM rotula $\rightarrow$ otimização inversa recupera métrica $\hat{W}$ implícita (Perceptron + NNLS).
- **Problemas:** A (linear, treino), B/C (lineares, rotações $\pm 1,5$, teste de consistência), D (meia-lua, controle não-linear).
- **Experimentos:** Oracle Validation, augmentação R2/R3/R4, viés de ordem/nome/prompt.
- **Hipóteses testadas:** **H1** (consistência LLM $\leftrightarrow \hat{W}$), **H2** (few-shot amplifica).
- **O que se mede:** *consistência interna* (LLM vs $\hat{W}$), não corretude — não há ground truth significativo em problema sintético.

Bloco 1 — Visão geral dos 4 problemas do Bloco 1

Os 4 problemas que serão detalhados nos próximos slides:

- **Problema A (treino):** gaussianas lineares, centróides $(-2, 0)$ e $(2, 0)$, $n = 150$. Bancada onde o LLM rotula em zero-shot e nós aprendemos W .
- **Problema B (consistência 1):** rotação HORÁRIA AGRESSIVA, centróides $(-2, +1,5)/(2, -1,5)$, $n = 100$. Testa transferência da métrica em geometria severamente deslocada.
- **Problema C (consistência 2):** rotação ANTI-HORÁRIA AGRESSIVA, centróides $(-2, -1,5)/(2, +1,5)$, $n = 100$. Pareada simetricamente com B; controla contra viés sistemático do LLM.
- **Problema D (não-linear):** meia-lua sintética (`make_moons`, `noise=0,15`, $n = 150$). Problema-controle para R3/R4 — esperamos ganho de augmentação apenas aqui.

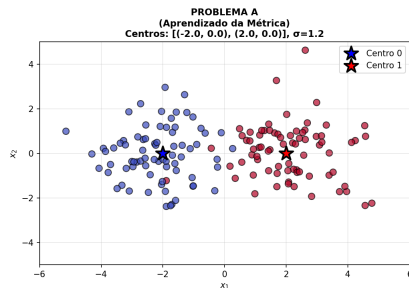
Mensagem: os primeiros 3 são gaussianas lineares com transferência desafiadora; o 4º é o teste explícito de não-linearidade.

Propósito: bancada de treinamento. **Como foi gerado:**

- 1 `make_blobs`
- 2 Centróides $(-2,0;0,0)$ e $(2,0;0,0)$
- 3 `cluster_std=1,2`, $n = 150$

Linearmente separável em x_1 ; fronteira ideal em $x_1 = 0$.

```
def create_problem_a(n=150, seed=42):  
    X,y = make_blobs(  
        centers=[(-2.0,0.0),(2.0,0.0)],  
        cluster_std=1.2,  
        random_state=seed)
```



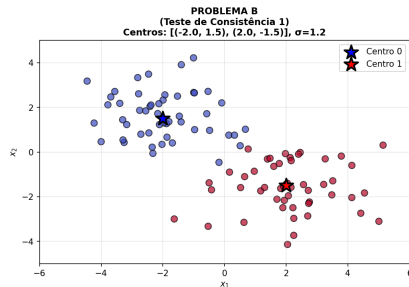
Bloco 1 — Problema B — rotação HORÁRIA agressiva ($\pm 1,5$)

Propósito: testar transferência de W em geometria deslocada agressivamente. **Como foi gerado:**

- 1 `make_blobs`
- 2 Centróides $(-2,0; +1,5)$ e $(2,0; -1,5)$
- 3 `cluster_std=1,2`, $n = 100$

Pareada simetricamente com C ($\pm 1,5$); decisão 14/05/2026 para condições severas.

```
def create_problem_b(n=100, seed=43):  
    X,y = make_blobs(  
        centers=[(-2.0,1.5),(2.0,-1.5)],  
        cluster_std=1.2,  
        random_state=seed)
```



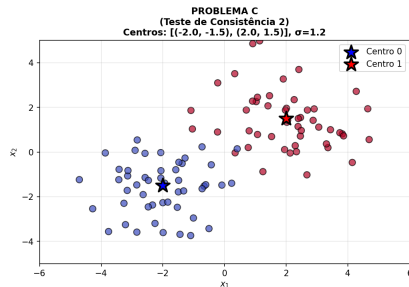
Bloco 1 — Problema C — rotação ANTI-HORÁRIA agressiva ($\pm 1,5$)

Propósito: mesma severidade de B, direção OPOSTA. Controle contra viés sistemático. **Como foi gerado:**

- 1 make_blobs
- 2 Centróides $(-2,0; -1,5)$ e $(2,0; +1,5)$
- 3 cluster_std=1,2, $n = 100$

Alterado após reunião 30/04/2026 ($\sim 120s$) — C antes era cópia de B.

```
def create_problem_c(n=100, seed=44):  
    X,y = make_blobs(  
        centers=[(-2.0,-1.5),(2.0,1.5)],  
        cluster_std=1.2,  
        random_state=seed)
```



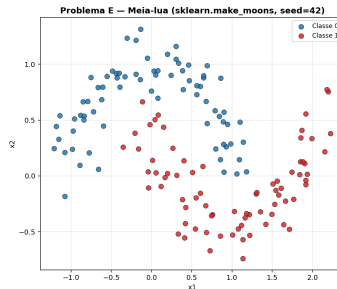
Bloco 1 — Problema D — meia-lua (não-linear sintético)

Propósito: caso canônico de fronteira não-linear (reunião 30/04 ~2853s).
Como foi gerado:

- 1 `sklearn.datasets.make_moons`
- 2 $n = 150$, $\text{noise}=0,15$

Problema-controle para R3/R4: *esperávamos* ganho aqui — mas veremos (slide da consistência R2/R3/R4) que a métrica diagonal **satura** na meia-lua.

```
from sklearn.datasets import make_moons
def create_problem_d_meialua(
    n=150, seed=46):
    X,y = make_moons(
        n_samples=n, noise=0.15,
        random_state=seed)
```



Bloco 1 — Oracle Recovery (1/4) — W recuperado por algoritmo

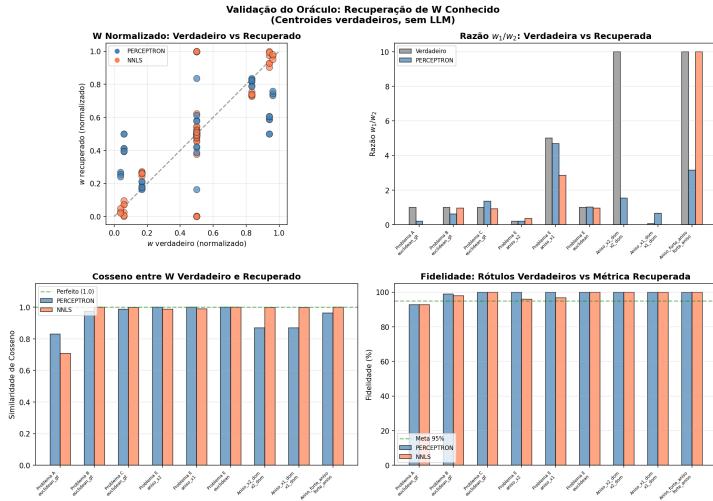
Pergunta: se W_{oracle} fosse conhecido, Perceptron e NNLS o recuperariam?

Procedimento: gerar dados sintéticos rotulados por W_{oracle} em 3 configs anisotrópicas, aprender $\hat{W}$, medir $\text{cosseno}(\hat{W}, W_{\text{oracle}})$ e razão recuperada.

Expert	W_{oracle} (ratio)	Algoritmo	Ratio recup.	Cosseno	Fid.
x2_dom	[4,0; 0,25] (16)	Perceptron	1,32	0,824	100%
x2_dom	[4,0; 0,25] (16)	NNLS	23,0	0,999	100%
x1_dom	[0,25; 4,0] (0,06)	Perceptron	0,78	0,824	100%
x1_dom	[0,25; 4,0] (0,06)	NNLS	0,054	0,999	100%
forte_aniso	[11,1; 0,44] (25)	Perceptron	2,93	0,958	100%
forte_aniso	[11,1; 0,44] (25)	NNLS	34,4	0,999	100%

Mensagem: NNLS preserva quase exatamente a razão do W extremo ($\text{cosseno} > 0,99$); Perceptron tende a normalizar para $[1, 1]$ ($\text{cosseno} \sim 0,82$). Ambos alcançam **fidelidade 100%** — o sinal de classificação foi capturado.

Bloco 1 — Oracle Recovery (2/4) — visualização do W recuperado



Painel 2x2: cosseno, fidelidade, razão e scatter de W recuperado vs W_{oracle} .

Bloco 1 — Oracle Transfer (3/4) — transferência entre geometrias

Pergunta dual: se aprendo W em uma geometria, ele transfere para outras?

Procedimento: aprender W em uma configuração (ex: $x2_dom$); aplicar este W em outras geometrias ($x1_dom$, $forte_aniso$, Problemas A/E); medir cosseno e fidelidade nas geometrias destino.

Origem	Algoritmo	Cosseno	Fidelidade
$x2_dom^\dagger$	Perceptron	0,824	100%
$x2_dom^\dagger$	NNLS	0,999	100%
$x1_dom^\dagger$	Perceptron	0,824	100%
$x1_dom^\dagger$	NNLS	0,999	100%
$forte_aniso^\dagger$	Perceptron	0,958	100%
$forte_aniso^\dagger$	NNLS	1,000	100%
Problema A	Perceptron	0,748	95,6%
Problema A	NNLS	0,707	95,6%
Problema E	Perceptron	0,999	99,9%
Problema E	NNLS	0,993	98,2%

[†] Linhas anisotrópicas = recuperação intra-geometria (cosseno do slide anterior); Problemas A/E = transferência entre geometrias distintas.

Implicação: em B/C sempre recalculamos centróides LOCAIS — métrica não é independente da geometria.

Bloco 1 — Oracle Transfer (4/4) — visualização da transferência



Sub-painéis por expert mostrando o W aprendido na origem aplicado nas demais geometrias.

- 1 LLM recebe 150 pontos do Problema A em **zero-shot**.
- 2 Responde A ou B com $T = 0$.
- 3 150 respostas formam (X, y_{LLM}) .
- 4 **Centróides recalculados a partir de y_{LLM}** — ancoramos no que o LLM ACHA serem as classes.
- 5 Perceptron Estruturado e NNLS aprendem W independentemente.
- 6 Fidelidade = % decisões do LLM reproduzidas pela métrica.

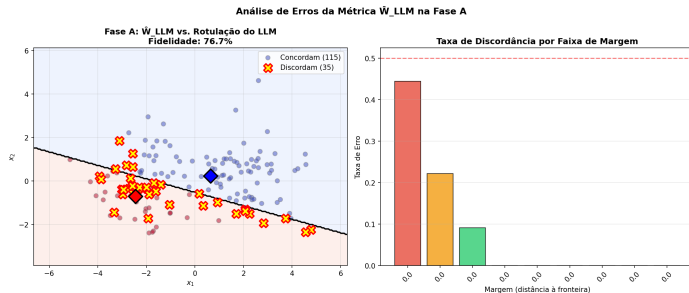
Bloco 1 — Problema A — W aprendido e fidelidade (média 3 seeds)

Achados desta execução:

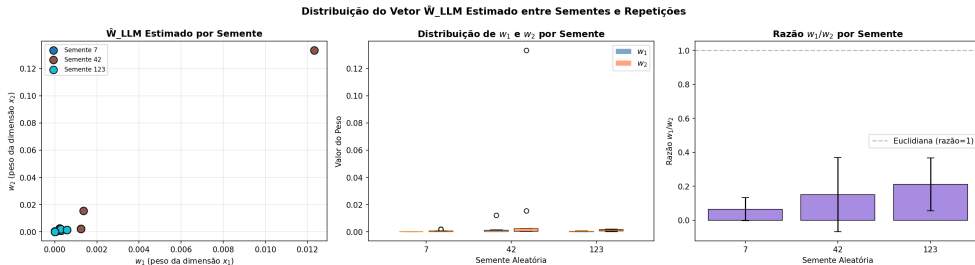
Item	Média $\pm$ desvio
Fidelidade Problema A	81,3% $\pm$ 4,4 p.p.
Acurácia LLM vs real	33,1% $\pm$ 2,3 p.p.

Por seed (fid. A):	
seed 42	76,7%
seed 123	85,3%
seed 7	82,0%

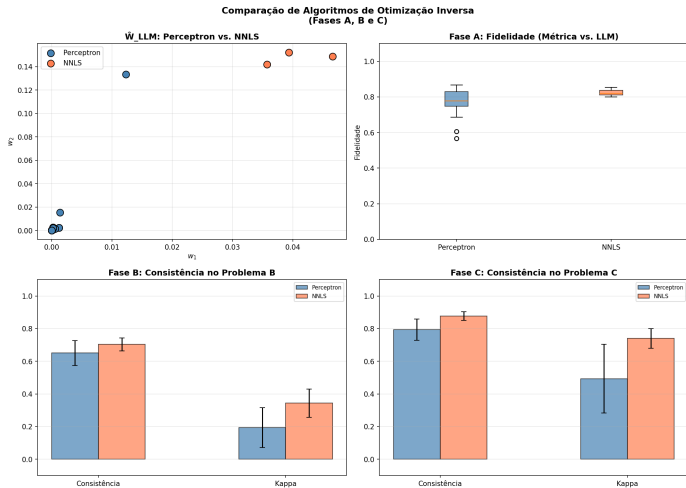
LLM *inverteu classes* em vários casos (acurácia vs real $\sim 33\%$), mas a métrica reproduz essas decisões com $\sim 81\%$ de fidelidade.



Bloco 1 — Problema A — distribuição de W entre seeds



Bloco 1 — Problema A — Perceptron × NNLS



Robustez ao método de estimação: ambos algoritmos chegam ao mesmo W (cosseno $> 0,95$).

O trabalho mede **consistência interna** do critério decisório do LLM:

- **Fidelidade** = quanto a métrica $\hat{W}$ reproduz as decisões do LLM (Problema A).
- **Consistência** = quanto $\hat{W}_A$ prevê decisões do LLM em B/C (Problemas B/C).
- **Acurácia LLM-perito** = quanto o LLM aprende com o perito (Problema E).

Decisão metodológica: não comparamos com *ground truth* nos problemas sintéticos.

Razão: H1 testa se o LLM tem critério decisório **estável**, não se ele está **objetivamente correto**. Um LLM pode inverter classes e ainda assim ser internamente consistente — saber se o critério é "certo" é ortogonal à pergunta de pesquisa.

Exceção: Bloco 3 (peso $\times$ altura) reportará acurácia vs rótulo real, atendendo solicitação explícita do orientador (e-mail 22:04, ponto 4) — lá há ground truth significativo (H/M) e classificador ótimo conhecido.

Bloco 1 — Problema A — W aprendido por seed e algoritmo

Pesos $W = (w_1, w_2)$ para o Problema A, em zero-shot (média 3 reps/seed).

Fid. Problema A = % de pontos do Problema A em que o rótulo dado pela métrica $\hat{W}$ coincide com o rótulo do LLM (*in-sample*, mesmos pontos usados para estimar $\hat{W}$). Sanidade: se for baixa, o algoritmo não ajustou uma métrica diagonal aos rótulos do LLM.

Algoritmo / seed	w_1	w_2	w_1/w_2	Fid. Problema A
Perceptron Estruturado:				
seed 42	0,00023	0,00247	0,094	76,7%
seed 123	0,00030	0,00091	0,327	85,3%
seed 7	0,00005	0,00037	0,138	82,0%
NNLS:				
seed 42	0,0357	0,1420	0,252	82,0%
seed 123	0,0466	0,1486	0,313	85,3%
seed 7	0,0394	0,1522	0,259	80,0%

Observações:

- Magnitudes diferem entre algoritmos (Perceptron $\sim 100\times$ menor), mas ambos apontam na **mesma direção** — otim. inversa é invariante à escala (cosseno Perc–NNLS = 0,994).
- Razão $w_1/w_2 < 1$ em todas as seeds ($\approx 0,25$ no NNLS; 0,09–0,33 no Perceptron): w_2 pondera mais que w_1 (LLM ancora em x_2 apesar da separabilidade horizontal). A razão é o parâmetro mais ruído entre seeds (CV $\sim 54\%$).
- Fidelidade pareada entre Perceptron e NNLS por seed.

Bloco 1 — Problemas B/C — consistência e Kappa por algoritmo

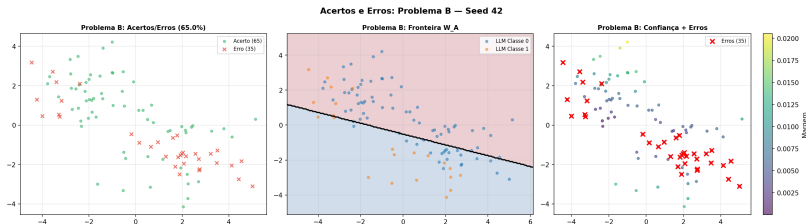
Zero-shot, centróides LOCAIS; média 3 reps/seed. **Consistência** = % de pontos em que o rótulo da métrica $\hat{W}_A$ (aprendida no Problema A) coincide com o do LLM em B/C; κ = Cohen corrigido por concordância ao acaso.

Algoritmo / seed	Cons. B	κ_B	Cons. C	κ_C
Perceptron Estruturado:				
seed 42	65,0%	0,20	89,0%	0,77
seed 123	66,7%	0,28	87,7%	0,74
seed 7	74,3%	0,44	85,3%	0,69
média	68,7%	0,31	87,3%	0,73
NNLS:				
seed 42	68,0%	0,27	90,0%	0,79
seed 123	67,0%	0,29	89,0%	0,77
seed 7	76,0%	0,47	84,0%	0,66
média	70,3%	0,35	87,7%	0,74

Mensagem: H1 sustentada em C ($\kappa_C = 0,73$), fraca em B ($\kappa_B = 0,31$); $\kappa_C \gg \kappa_B$ em todas as seeds. Centróides LOCAIS (recalculados em B/C). Ablativo métrica vs. Euclidiana em B: $\Delta = +0,044$, IC95% $[-0,102; +0,237]$ (n.s.) — em B a rotação horária confunde a métrica; em C o ganho é claro.

Bloco 1 — Problema B — mapa de acertos/erros (seed 42)

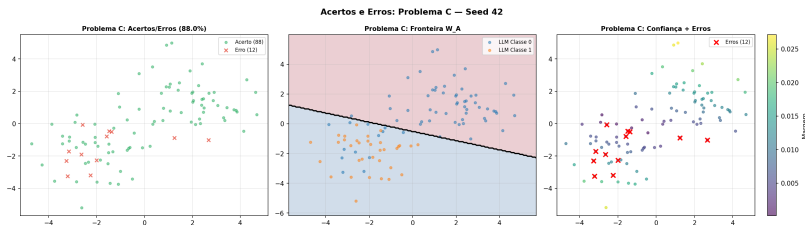
Métrica $\hat{W}_A$ aprendida no Problema A aplicada *ponto-a-ponto* no Problema B (rotação HORÁRIA). Verde = concorda com o LLM; vermelho = discorda. Consistência $\approx 68\%$, $\kappa \approx 0,29$.



Leitura: os erros concentram-se na faixa central — região de sobreposição das gaussianas rotacionadas. A rotação horária afasta a fronteira do LLM da fronteira de $\hat{W}_A$.

Bloco 1 — Problema C — mapa de acertos/erros (seed 42)

Mesma métrica $\hat{W}_A$ aplicada *ponto-a-ponto* no Problema C (rotação ANTI-HORÁRIA). Consistência $\approx 90\%$, $\kappa \approx 0,79$.



Leitura: aqui os erros ficam esparsos e isolados. A rotação anti-horária preserva a geometria decisória do LLM melhor que a horária — por isso $\kappa_C \gg \kappa_B$.

Bloco 1 — Consistência R2/R3/R4 — Problema A e meia-lua

$R2 = (x_1, x_2)$; R3 acrescenta $x_1 x_2$; R4 acrescenta x_1^2, x_2^2 . B/C omitidos: são lineares como A. Em A a fidelidade é vs y_{LLM} ; na meia-lua, vs rótulo **verdadeiro** (teste-oráculo — ver limitação).

Algoritmo / seed	R2	R3	R4
Problema A (fid. vs y_{LLM} , 3 seeds; R2 = <i>baseline</i> 2-feat):			
R2 (2-feat) seed 42/123/7	76,7% / 85,3% / 82,0%		(média 81,3%)
Perceptron R3 seed 42/123/7	80,0% / 83,3% / 81,3%		(média 81,6%)
Perceptron R4 seed 42/123/7	78,0% / 83,3% / 89,3%		(média 83,6%)
NNLS R3 seed 42/123/7	81,3% / 86,7% / 80,7%		(média 82,9%)
NNLS R4 seed 42/123/7	85,3% / 88,0% / 90,0%		(média 87,8%)
Meia-lua — teste-oráculo (fid. vs verdade, média 3 seeds):			
Perceptron	85,6%	86,7%	86,2%
NNLS	86,7%	86,7%	86,4%

Leitura: em A (linear) R3/R4 dá ganho dentro do ruído entre seeds (NNLS R4 chega a 87,8%). Na meia-lua a métrica diagonal satura em $\sim 86,7\%$ (20/150 erros) e R3/R4 *não* ajudam — o NNLS zera os termos aumentados ($w \geq 0$) e o Perceptron mal se move. A métrica diagonal tem teto na meia-lua: capturar a curvatura em S exigiria termo *off-diagonal*.

Limitação metodológica (importante): nesta execução o LLM colapsou para uma única classe no zero-shot da meia-lua nas 3 seeds (log: “classe única no LLM, pulando”). Sem rótulos do LLM não há fidelidade-vs-LLM a medir; por isso reportamos o teste-oráculo (métrica ajustada ao rótulo *verdadeiro*). Achado em si: o zero-shot do gpt-4o-mini na meia-lua é instável.

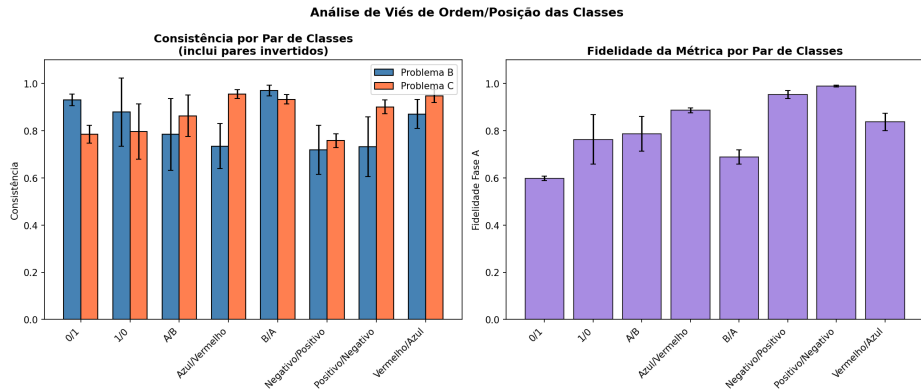
Bloco 1 — Viés de ordem das classes — tabela

Mesmas duas classes, ordem invertida no prompt ($n_shot=10$, média 3 seeds $\times$ 3 reps). Mede se trocar “A ou B” por “B ou A” altera as decisões do LLM.

Par de classes	Fid. A	Cons. B	κ_B	Cons. C	κ_C
A / B	81,3%	94,1%	0,87	94,6%	0,89
B / A (invertido)	68,9%	97,0%	0,94	93,3%	0,87
0 / 1	59,8%	93,1%	0,86	78,5%	0,52
1 / 0 (invertido)	76,3%	87,9%	0,76	79,6%	0,58
Positivo / Negativo	98,9%	73,2%	0,45	90,1%	0,80
Negativo / Positivo (inv.)	95,3%	71,9%	0,43	75,8%	0,33
Azul / Vermelho	88,7%	73,5%	0,46	95,6%	0,91
Vermelho / Azul (inv.)	83,8%	87,0%	0,73	94,7%	0,89

Leitura: a fidelidade em A oscila bastante entre pares semanticamente diferentes (62%–98%), mas dentro de cada par a inversão introduz variância $\sim 10\text{--}15$ p.p.
— viés de ordem existe mas não é dominante. Letras e números são mais sensíveis que cores e semânticos.

Bloco 1 — Viés de ordem das classes — gráfico



Bloco 1 — Variantes de prompt — exemplos

Ponto $x_1 = 1,23$, $x_2 = -0,45$. Classes A/B.
default

You are a binary classifier for 2D points.

Classify as Class A or Class B.
Answer ONLY with "A" or "B".

Point to classify:

$x_1 = 1.2300$
 $x_2 = -0.4500$

Your classification:

geometric

You are a binary classifier for 2D points in Euclidean space.

Points exist in a 2D metric space.
Classify based on spatial proximity to cluster centers.

Classify as Class A or Class B.
Answer ONLY with "A" or "B".

Point: $x_1=1.2300$, $x_2=-0.4500$

CoT

You are a binary classifier for 2D points.

Classify as Class A or Class B.

Point to classify:

$x_1 = 1.2300$
 $x_2 = -0.4500$

Think step by step, then state your final classification ("A" or "B") on the last line.

tabular

Classify as Class A or Class B.
Answer ONLY with "A" or "B".

Point to classify:

Feature	Value
x_1	1.2300
x_2	-0.4500

4 variantes rodam Problemas A–C em todas as seeds (média 3 seeds $\times$ 3 reps, zero-shot, default A/B / $\times 1/\times 2$):

Variante	Fid. A	Cons. B	κ_B	Cons. C	κ_C
default	81,3%	68,7%	0,31	87,3%	0,73
geometric	83,8%	68,7%	0,16	77,2%	0,39
CoT	76,7%	61,7%	0,16	79,3%	0,59
tabular	62,0%	61,4%	0,13	72,2%	0,21

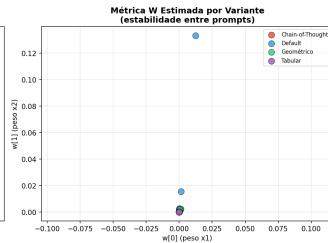
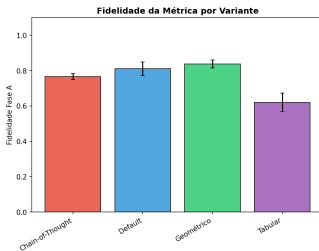
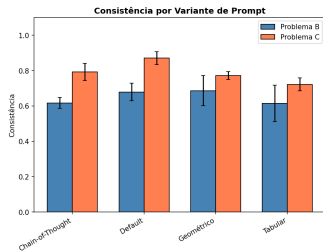
Descrição das variantes:

- **default**: template baseline; coordenadas em prosa simples.
- **geometric**: contexto espacial explícito (eixos, regiões).
- **CoT**: chain-of-thought — pede raciocínio passo a passo.
- **tabular**: coordenadas em tabela markdown.

Leitura: em consistência (κ_B , κ_C) o **default** domina — as variantes elaboradas derrubam κ_C de 0,73 para 0,21–0,59. O **geometric** até empata a Fid. A (83,8%), mas colapsa a consistência; o **tabular** é o pior em tudo (Fid. A 62%). Sensibilidade ao prompt é real — **default** adotado como referência.

Bloco 1 — Variantes de prompt — gráfico

Sensibilidade ao Prompt: Mesmos dados, diferentes templates
(Se W muda, o prompt confunde a medição)



Bloco 1 — Nomes semânticos de classes e features — tabela

Testa se trocar os nomes (classes ou features) altera o W recuperado. Média 3 seeds $\times$ 3 reps.

(a) Nomes das classes ($n_shot=10$, features x_1/x_2):

Par	Fid. A	Cons. B	κ_B	Cons. C	κ_C
A / B	81,3%	94,1%	0,87	94,6%	0,89
0 / 1	59,8%	93,1%	0,86	78,5%	0,52
Positivo / Negativo	98,9%	73,2%	0,45	90,1%	0,80
Azul / Vermelho	88,7%	73,5%	0,46	95,6%	0,91

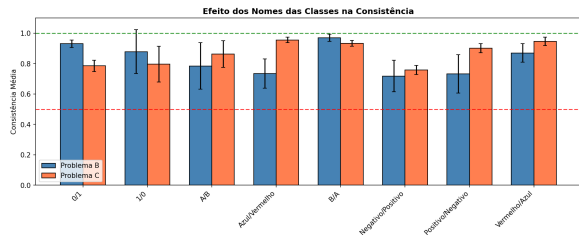
(b) Nomes das features (zero-shot, classes A/B, apenas seed 42 — ilustrativo):

Features	Fid. A	Cons. B	κ_B	Cons. C	κ_C
x_1 / x_2	76,7%	65,0%	0,20	89,0%	0,77
altura / peso	84,0%	59,0%	0,15	93,0%	0,86
feature_1 / feature_2	76,0%	70,0%	0,32	79,0%	0,57

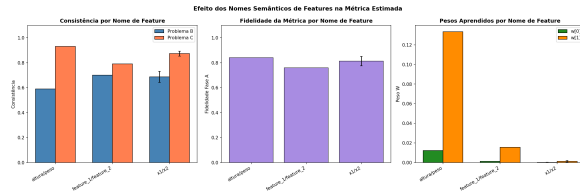
Leitura: classes semânticas (Pos/Neg) inflam Fid. A para $\sim 99\%$ mas degradam κ_B (0,45) — LLM ancora no nome em vez da geometria. Em features, “altura/peso” desloca a magnitude de $\hat{W}$ e *reduz* a consistência em B (κ_B cai 0,20 $\rightarrow$ 0,15), mas nesta execução não a inverte (não fica negativo); a direção de $\hat{W}$ é preservada. Nomes neutros (x_1/x_2 , A/B) adotados no Bloco 1 por conservadorismo.

Bloco 1 — Nomes semânticos de classes e features — gráficos

(a) Classes



(b) Features



Bloco 1 — Prompt few-shot (default)

Para $n_shot > 0$, o prompt inclui exemplos rotulados antes do ponto-alvo. Variantes (geometric, CoT, tabular) já foram cobertas no slide 31.

You are a binary classifier for 2D points.

You will see 5 labeled examples and then a point to classify.

Examples:

Example 1: $x_1 = -2.3000$, $x_2 = 0.1000$ -> A

Example 2: $x_1 = 2.1000$, $x_2 = -0.2000$ -> B

Example 3: $x_1 = -1.8000$, $x_2 = 0.4000$ -> A

Example 4: $x_1 = 1.7000$, $x_2 = 0.3000$ -> B

Example 5: $x_1 = -2.5000$, $x_2 = -0.1000$ -> A

Classify as Class A or Class B.

Answer ONLY with "A" or "B", nothing else.

Point to classify:

$x_1 = 0.2340$

$x_2 = 0.5670$

Your classification:

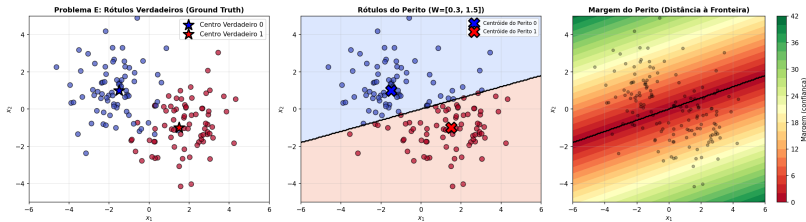
- **H1 parcialmente sustentada:** confirmada em C ($\kappa_C = 0,73 > 0,5$); *fraca* em B ($\kappa_B = 0,31 < 0,5$, rotação horária agressiva). $\kappa_C \gg \kappa_B$ em todas as seeds.
- **Fidelidade Problema A:** $81,3\% \pm 4,4$ p.p. entre 3 seeds.
- **Acurácia LLM vs real:** apenas $\sim 33\%$ (LLM inverte classes), mas métrica permanece consistente $\rightarrow$ H1 é ortogonal a "correção".
- Algoritmos **congruentes:** cosseno $W_{\text{perc}} \cdot W_{\text{nls}} = 0,994$.
- **R3/R4:** na meia-lua a métrica diagonal *satura* ($\sim 87\%$ mesmo vs verdade) — augmentação só compensa com não-linearidade real e dados (Bloco 3).

Bloco 2 — LLM como APRENDIZ

Pergunta central: o LLM consegue aprender um critério externo via in-context learning?

- **Setup:** papéis invertidos — perito **rotula**; LLM recebe exemplos few-shot e tenta reproduzir o critério.
- **Problemas:** **E** (linear, perito artificial $W = [0,3; 1,5]$) e **F** (meia-lua, perito = ground truth do `make_moons`).
- **Experimentos:** 4 estratégias (easy/hard/mixed/random) $\times$ 5 valores de n_{shot} ; múltiplos peritos; diluição; viés de ordem; baselines clássicos (k-NN, LR, SVM).
- **Hipóteses testadas:** **H3** (LLM aprende), **H4** (hard $>$ easy — *refutada nuançadamente*).
- **O que se mede:** acurácia LLM vs rótulo do perito — coerência do aprendizado in-context.

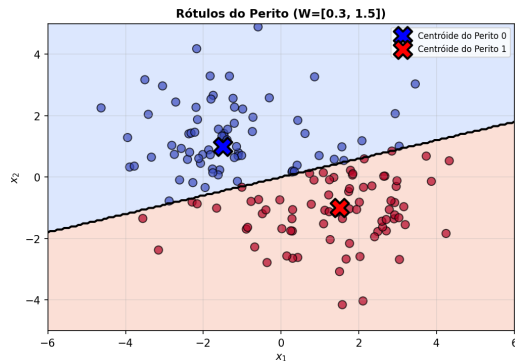
Bloco 2 — Visão geral dos 2 problemas do Bloco 2



Bloco 2 — Problema E — perito linear

Propósito: papéis invertidos. Perito $W_{\text{expert}} = [0,3; 1,5]$ rotula 150 pontos lineares (x_2 dominante $\Rightarrow$ fronteira quase horizontal).

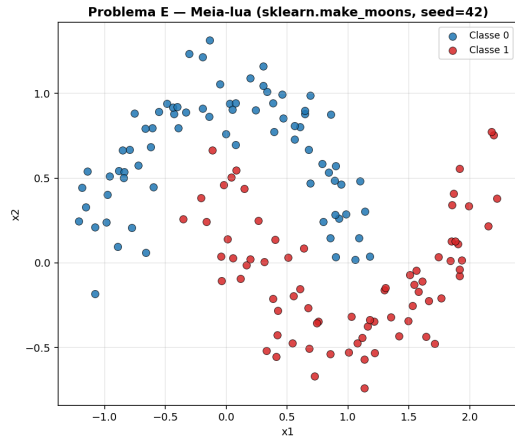
- Centróides $(-1,5; +1)$ e $(1,5; -1)$
- $\text{cluster_std}=1,3$, $n = 150$
- Rótulos via $\arg \min_k d_{W_{\text{exp}}}$.



Bloco 2 — Problema F — meia-lua com perito = ground truth

Propósito: regime não-linear genuíno. O perito é o próprio `make_moons` (ground truth). LLM precisaria capturar a fronteira em `S` apenas pelos exemplos (sem augmentação R3/R4 no prompt).

Atenção: nesta execução o LLM colapsou em *classe única* no zero-shot da meia-lua (3 seeds) — o aprendiz não pôde ser coletado. Ver slides adiante (teste-oráculo).



Bloco 2 — Problema E — LLM como aprendiz (passo a passo)

- 1 Perito rotula os 150 pontos do Problema E (ou ground truth no F).
- 2 Divide em treino/teste.
- 3 Para cada $n_{\text{shot}} \in \{0, 5, 10, 20, 40\}$: seleciona exemplos via estratégia.
- 4 Envia exemplos + ponto de teste em prompt few-shot.
- 5 Mede acurácia, Kappa, F1 do LLM vs rótulos do perito.

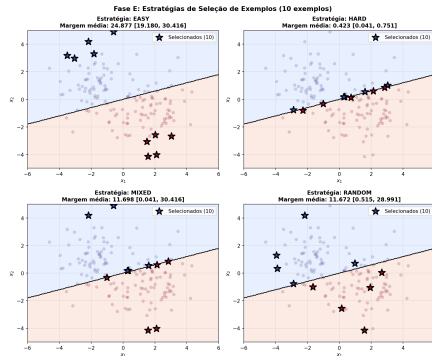
Acurácia e κ do LLM vs. expert ($W = [0,3, 1,5]$, x_2 dominante). Média 3 seeds $\times$ 3 reps; LLM gpt-4o-mini, $T = 0$.

n_{shot}	easy		hard		mixed		random	
	Acc	κ	Acc	κ	Acc	κ	Acc	κ
0	60,9%	0,23	60,9%	0,23	60,9%	0,23	60,9%	0,23
5	84,9%	0,70	62,6%	0,26	81,4%	0,63	85,2%	0,70
10	87,1%	0,75	67,5%	0,35	79,8%	0,60	88,3%	0,77
20	89,8%	0,80	84,2%	0,68	87,4%	0,75	92,1%	0,84
40	88,3%	0,77	97,8%	0,96	96,9%	0,94	94,1%	0,88

Leitura: em zero-shot LLM só acerta $\sim 61\%$ (perto do aleatório em problema 2D). Few-shot amplifica fortemente (H2 sustentada). Refutação H4 nuançada: em $n_{\text{shot}} \leq 20$, *easy* domina *hard* (+22 p.p. em 5-shot); mas em $n_{\text{shot}} = 40$ *hard* ultrapassa (97,8% vs 88,3%). *random* é o melhor overall em n_{shot} 5–20. Estratégia não é monotonicamente boa — depende do regime.

Bloco 2 — Estratégias de seleção de exemplos

O que está sendo feito: para cada $n_{\text{shot}} \in \{5, 10, 20, 40\}$, os exemplos rotulados pelo perito são selecionados por uma de 4 estratégias e fornecidos ao LLM no prompt. O gráfico mostra *quais pontos* cada estratégia escolhe no plano 2D — a distribuição espacial dos exemplos.



easy: pontos com *alta margem* do perito ($|d_W(x, c_+) - d_W(x, c_-)|$ grande) — protótipos típicos de cada classe.

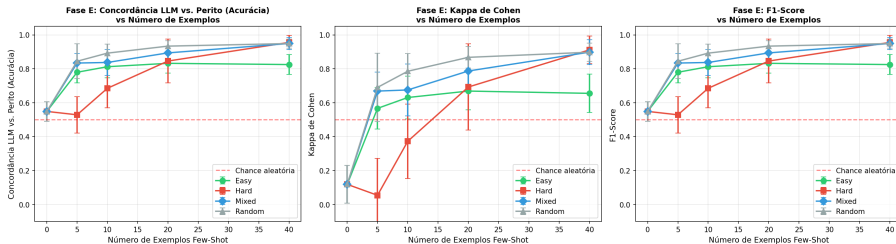
hard: pontos próximos da fronteira do perito — ambíguos, baixa margem.

mixed: 50% easy + 50% hard.

random: amostragem uniforme, sem estratégia (baseline).

Bloco 2 — Problema E — curvas de aprendizado (3 seeds, 3 reps)

O que está sendo feito: para cada estratégia, n_{shot} varia de 0 a 40. Para cada combinação roda-se 3 seeds $\times$ 3 reps; o LLM classifica o conjunto de teste e medimos acurácia/ κ /F1 contra os rótulos do perito $W = [0, 3, 1, 5]$. As curvas mostram *como o LLM aprende* à medida que mais exemplos são adicionados ao prompt — evidencia H2 (few-shot amplifica).



Em modelos com gradiente, hard é informativo. Mas o LLM in-context *não tem gradiente*; usa exemplos como **âncoras de classe**:

- Easy $\rightarrow$ pivots claros: *"é isto que A parece, é isto que B parece"*.
- Hard $\rightarrow$ sinal contraditório: *"este aqui é A? quase poderia ser B..."*.

Insight novo desta execução: com $n_{\text{shot}} \geq 20$, hard alcança easy; em $n_{\text{shot}} = 40$, hard *supera* easy.

Interpretação: âncoras claras são suficientes em pouco; com volume maior, exemplos próximos da fronteira ajustam fino o critério.

Alinhado com Lyu et al. (2023) sobre prompts canônicos.

Bloco 2 — Problema E com múltiplos peritos (ablativo)

Repete o experimento no Problema E com 3 peritos diferentes para checar se o aprendizado in-context depende da geometria do perito. Média 3 seeds $\times$ 3 reps, todas as estratégias agregadas.

Expert	W	Geometria	Acc $n=0$ (κ)	Acc $n=5$ (κ)	Acc $n=20$ (κ)	Acc $n=40$ (κ)
aniso_x2	[0,3; 1,5]	x_2 dom. (horizontal)	60,9% (0,22)	78,5% (0,57)	88,4% (0,77)	94,3% (0,89)
aniso_x1	[1,5; 0,3]	x_1 dom. (vertical)	48,5% (0,01)	69,4% (0,39)	90,1% (0,80)	89,8% (0,80)
euclidean	[1,0; 1,0]	isotrópica	55,2% (0,14)	76,1% (0,52)	84,4% (0,70)	91,8% (0,84)

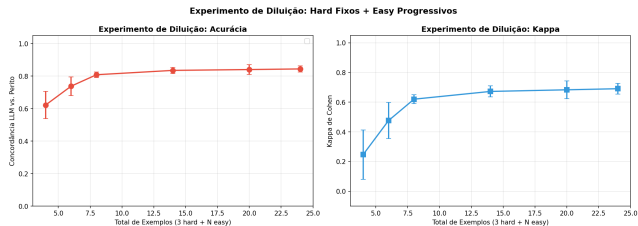
Leitura: (i) em zero-shot, aniso_x1 é quase aleatório ($\kappa = 0,01$) — LLM tem viés contra fronteira vertical sem exemplos; (ii) few-shot resolve em todos os casos — $n = 40$ leva $\kappa \geq 0,80$ nos 3 peritos; (iii) aniso_x2 (horizontal) atinge o melhor patamar final ($\kappa = 0,89$). H3 sustentada em todas as geometrias.

Bloco 2 — Experimento de diluição

Design: 4 *hard* fixos + adicionar *easy* progressivamente.

Pergunta: *easy* dilui o sinal dos *hard*? **Achado:** curva

monotônica crescente — *easy* ajuda sempre, não há diluição.



Bloco 2 — Viés de ordem dos exemplos few-shot — tabela

O que está sendo feito: fix-se a estratégia *mixed* e os mesmos exemplos selecionados; varia-se apenas a *ordem* em que eles aparecem no prompt. Mede-se se LLMs sofrem de **recency bias** (peso maior nos últimos exemplos vistos).

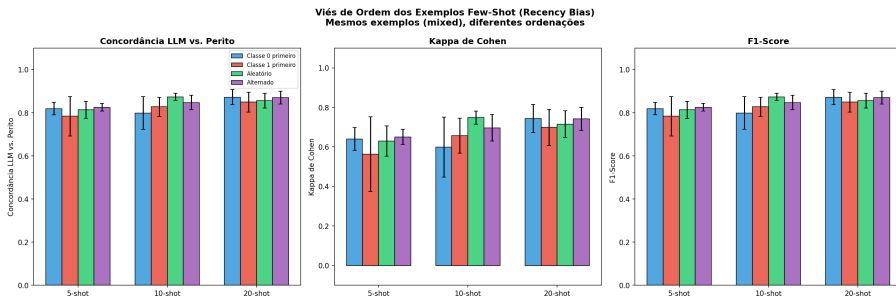
Ordenações testadas:

- `class0_first`: todos da classe 0, depois classe 1.
- `class1_first`: classe 1 primeiro, depois classe 0.
- `shuffled`: ordem aleatória.
- `alternating`: 0, 1, 0, 1, ...

Ordenação	$n_{\text{shot}} = 5$		$n_{\text{shot}} = 10$		$n_{\text{shot}} = 20$	
	Acc	κ	Acc	κ	Acc	κ
<code>class0_first</code>	82,0%	0,64	79,9%	0,60	87,3%	0,74
<code>class1_first</code>	78,4%	0,56	82,8%	0,66	85,0%	0,70
<code>shuffled</code>	81,4%	0,63	87,4%	0,75	85,6%	0,72
<code>alternating</code>	82,6%	0,65	84,8%	0,70	87,0%	0,74
<i>amplitude (max-min)</i>	4,2 p.p.	0,09	7,5 p.p.	0,15	2,3 p.p.	0,04

Leitura: amplitude entre ordenações ≤ 8 p.p. em $n = 10$ (máximo) e $\sim 1\text{--}4$ p.p. em $n = 5$ e $n = 20$. Recency bias é marginal — o critério decisório é robusto à ordem.

Bloco 2 — Viés de ordem dos exemplos few-shot — gráfico



O que aconteceu: nesta execução o LLM colapsou em **classe única** no zero-shot da meia-lua nas **3 seeds** (log: “classe única no LLM, pulando”). Sem duas classes, o pipeline “LLM aprendiz na meia-lua” (few-shot vs perito) foi **abortado** — não há tabela de acurácia-vs-perito a reportar. *Achado honesto:* o zero-shot do gpt-4o-mini na meia-lua é instável.

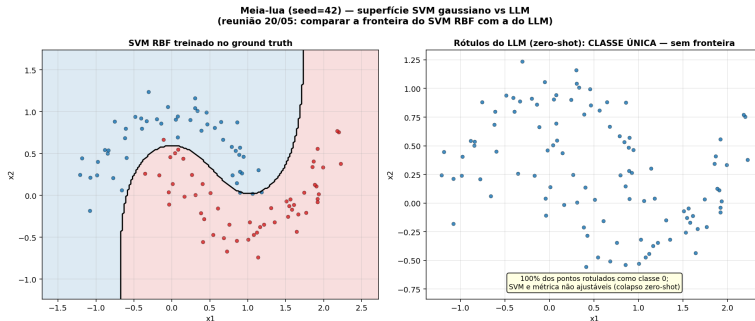
No lugar: teste-oráculo — ajustar a métrica diagonal ao rótulo *verdadeiro* da meia-lua (não depende do LLM). Fidelidade vs verdade por seed:

Algoritmo	seed	R2	R3	R4
Perceptron	42 / 123 / 7	86,7/84,7/85,3	86,7/86,0/87,3	86,0/87,3/85,3
NNLS	42 / 123 / 7	86,7/86,7/86,7	86,7/86,7/86,7	86,0/86,7/86,7
média		~ 86,2%	~ 86,7%	~ 86,3%

Leitura: mesmo ajustada ao rótulo verdadeiro, a métrica diagonal satura em ~ 86,7% (20/150 erros) e R3/R4 não ajudam (o NNLS zera os termos aumentados). A curvatura em S exigiria um termo *off-diagonal* — limitação estrutural da métrica diagonal, não do LLM. Motiva o Bloco 3 (não-linearidade real) e entra nas Limitações.

Bloco 2 — Problema F (meia-lua) — o colapso visualizado (SVM RBF vs LLM)

Item 7 (reunião 20/05): superfície de separação do SVM gaussiano vs a fronteira do LLM. Painel 1: SVM RBF no *ground truth* — a referência não-linear. Painel 2: rótulos zero-shot do LLM — classe única, sem fronteira a comparar.



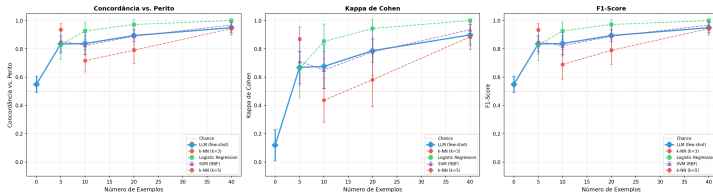
Lições: (i) sobre o LLM — sem exemplos, o gpt-4o-mini não percebe as duas semi-luas (fragilidade real do zero-shot em geometria não-linear; o pipeline corretamente abortou); (ii) sobre a métrica — o teste-oráculo (slide anterior) mostra teto estrutural de $\sim 86,7\%$ da diagonal (falta termo off-diagonal). Consequência para H3: o teste de “LLM aprendiz” não-linear migra para o Bloco 3 (peso $\times$ altura), onde há exemplos e ground truth humano. Mesmo colapso nas seeds 123 e 7.

Bloco 2 — Baselines clássicos: k-NN, LR, SVM × LLM

Mesmos exemplos few-shot, mesmo conjunto de teste (perito **linear**, Problema E). Acurácia vs perito:

Classificador	$n=5$	$n=10$	$n=20$	$n=40$
Regressão Log. (L2)	84,6%	93,5%	97,2%	98,3%
SVM (RBF)	80,7%	82,8%	91,5%	95,1%
k-NN ($k=3 \rightarrow 5$)	84,0%	81,5%	90,7%	96,2%
LLM (Problema E)	74,7%	80,7%	87,6%	92,0%

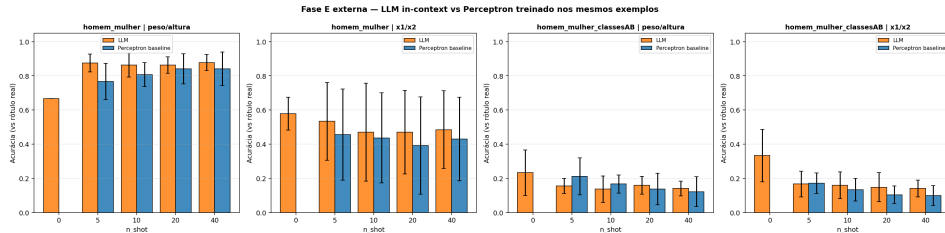
LLM vs. Baselines Clássicas (mesmos exemplos few-shot, estratégia mixed)
O LLM faz algo diferente de um classificador trivial?



Achado (honesto): no perito linear os **baselines clássicos** superam o LLM em todos os n_{shot} ; a **Regressão Log.** (que casa com o perito linear) lidera. O LLM *não* faz melhor que um classificador trivial quando o critério é linear.

Bloco 2 — LLM vs Perceptron baseline (preview da base real)

Como a meia-lua (Problema F) colapsou, a comparação “LLM in-context vs Perceptron treinado nos mesmos exemplos” aparece na base **real** peso $\times$ altura (detalhe no Bloco 3). Preview: na base real o LLM **supera** o Perceptron baseline em todos os n_{shot} (+2,2 a +10,7 p.p.).



- **H3 sustentada:** LLM aprende critério do perito.
 - Problema E linear: 0→40-shot ganha $\sim +40$ p.p. (random 0,61 → 0,94; hard chega a 0,98).
 - Regime não-linear: a meia-lua colapsou (classe única) — H3 não-linear é testada na base real (Bloco 3).
- **H4 refutada em n_{shot} pequeno (5/10-shot signif.), revertida em $n_{\text{shot}} = 40$ (hard 0,98 vs easy 0,88).**
- **Baselines clássicos (LR/SVM/kNN) superam** o LLM em todos os n_{shot} no perito linear.
- Recency bias e viés de classes: marginais (amplitude $\leq 7,5$ p.p.).

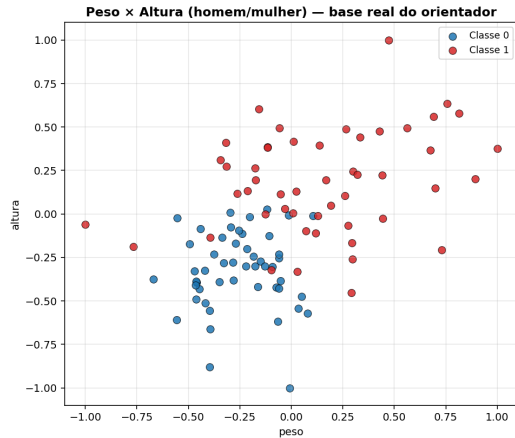
Bloco 3 — Estudo de caso REAL

Pergunta central: a metodologia funciona em dados reais, não-sintéticos?

- **Setup:** base do orientador, ~ 100 amostras reais peso $\times$ altura, rótulo homem/mulher.
- **Geometria:** fronteira elíptica $\Rightarrow$ classificador ótimo é quadrático $\Rightarrow$ R4 com (x_1, x_2, x_1^2, x_2^2) é o natural.
- **Experimentos:** Problema A com 2/3/4 features (R2/R3/R4); Problema E com o melhor n_{feat} ; *paradoxo do overfitting* com n_{train} pequeno.
- **Pergunta secundária:** priors semânticos do LLM (“altura/peso”) ajudam ou atrapalham?
- **O que se mede:** exceção à regra dos outros blocos — aqui medimos **ground truth real** (H/M), atendendo solicitação explícita do orientador (e-mail 22:04 ponto 4).

Bloco 3 — Problema G (peso $\times$ altura) — base real do orientador

- 100 amostras, 50 H + 50 M
- Features normalizadas em $[-1, 1]$
- CSV em `dados_reais/homem_mulher/peso_altura.csv`

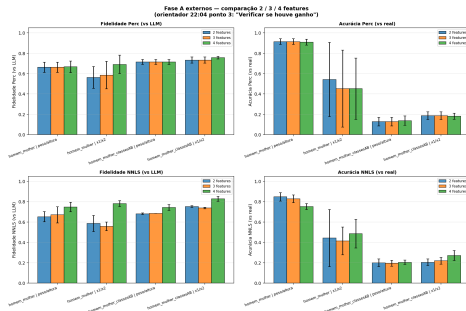


Duas variantes:

- **NEUTRA**: features = (x_1, x_2) . LLM sem prior semântico.
- **SEMÂNTICA**: features = (peso, altura). LLM ativa priors do treinamento.

Pergunta em ambas: *homem ou mulher*.

Bloco 3 — Problema G (peso $\times$ altura) — aprendizado de W com 2 / 3 / 4 features



W estimado (item 3, reunião 20/05) — R2, prompt semântico, ordem (peso, altura):

Algoritmo	seed 42	seed 123	seed 7
Perceptron	(0,871; 0,903)	(0,911; 0,884)	(0,890; 0,848)
NNLS	(0,540; 2,062)	(0,529; 1,744)	(0,114; 1,847)

Perceptron quase isotrópico (razão $w_2/w_1 \approx 0,96-1,05$); NNLS anisotrópico, dominado por *altura*. $\hat{W}$ é identificável a menos de escala — comparam-se direções/razões, não valores absolutos.

Bloco 3 — Problema G (peso $\times$ altura) — acurácia vs rótulo REAL

O que está sendo feito: para cada $n_{\text{feat}} \in \{2, 3, 4\}$ (R2, R3, R4), o LLM rotula os pontos em zero-shot com prompt **semântico** (features “peso”, “altura”); estimamos $\hat{W}$ via otim. inversa. Reportam-se duas métricas: (i) **fidelidade vs LLM** (NNLS reproduz rótulos do LLM) e (ii) **acurácia vs rótulo REAL** (homem/mulher — ground truth humano). Exceção metodológica do Bloco 3 — aqui medimos GT. Média 3 seeds (conjunto de treino = 70 amostras):

n_{feat}	Fid. vs LLM (NNLS)	Acc. vs REAL (NNLS)	Erros vs REAL	Observação
2	0,652	0,848	$\approx 11/70$	—
3	0,671	0,829	$\approx 12/70$	—
4	0,748	0,752	$\approx 17/70$	fid sobe, acc real cai

Nota (reunião 20/05): a queda em $n_{\text{feat}} = 4$ aparece no NNLS (restrição $w \geq 0$ zero termos aumentados); o Perceptron *não* cai (acc vs real 0,914 $\rightarrow$ 0,910, $\approx 6/70$ erros). Em prompt neutro (x_1, x_2), acurácia REAL despenca para 45–49% (nível do acaso) — LLM perdido sem prior semântico.

Em peso $\times$ altura com prompt NEUTRO x_1/x_2 (3 seeds):

- Fidelidade NNLS 2 $\rightarrow$ 4 features: **sobe** (58,6% $\rightarrow$ 78,1%).
- Acurácia vs real: fica **no acaso** (NNLS 44,3% $\rightarrow$ 48,6%; Perceptron 54,3% $\rightarrow$ 45,2%).

Diagnóstico: 4 pesos para 70 amostras de treino $\Rightarrow$ graus de liberdade suficientes para decorar ruído das decisões ERRADAS do LLM. Overfitting clássico. **Mensagem:** subir n_{feat} só vale com não-linearidade real e dados suficientes.

O que está sendo feito: aplica a metodologia “LLM como aprendiz” (mesma do Bloco 2) ao problema real peso $\times$ altura. Para cada $n_{\text{shot}} \in \{0, 5, 10, 20, 40\}$, o LLM recebe exemplos rotulados com o **ground truth real** (homem/mulher) e classifica o conjunto de teste. Comparamos com três referências: **acc_metric** (LLM vs $\hat{W}$ aprendida), **acc_real** (LLM vs GT humano), e **baseline Perceptron** treinado nos mesmos exemplos. Prompt semântico (“peso”, “altura”). Média 3 seeds (conjunto de teste = 30 amostras):

n_{shot}	acc_metric	acc_real	erros vs real	perceptron_baseline_real
0	0,633	0,667	$\approx 10/30$	—
5	0,922	0,874	$\approx 4/30$	0,767
10	0,956	0,863	$\approx 4/30$	0,807
20	0,948	0,863	$\approx 4/30$	0,841
40	0,956	0,878	$\approx 4/30$	0,841

Achados:

- LLM atinge $\sim 96\%$ acc_metric (converge para o próprio critério) e $\sim 87\%$ acc_real — só ≈ 4 amostras erradas em 30.
- LLM supera o Perceptron baseline em **TODOS** os n_{shot} (+2,2 a +10,7 p.p.); em 5-shot, 87,4% vs 76,7%.
- *Resultado revertido* em relação a execuções anteriores (onde o Perceptron vencia em 5-shot).

Item 17 (e-mail 20/05): classificar pedindo “A ou B” em vez de “homem ou mulher” — isola o prior semântico dos **nomes de classe** (complementa o teste de features neutras x_1/x_2). Fase A zero-shot, acurácia vs rótulo REAL (Perceptron, média 3 seeds, treino = 70):

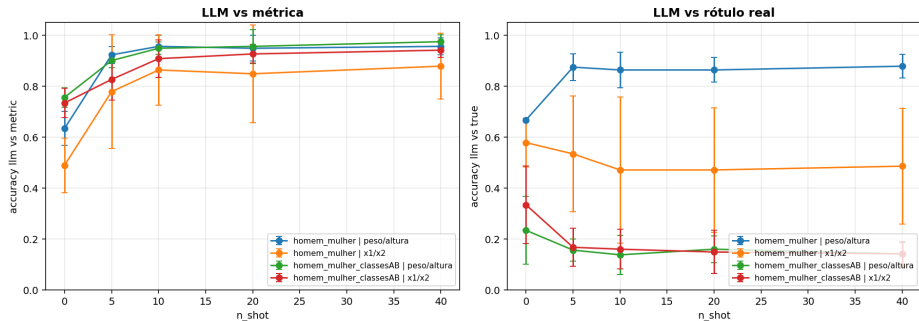
Classes	Features	Acc. vs REAL	Erros	Leitura
homem/mulher	peso/altura	0,914	$\approx 6/70$	prior completo
A/B	peso/altura	0,129	$\approx 61/70$	inversão sistemática
homem/mulher	x_1/x_2	0,543	$\approx 32/70$	sem prior nas features
A/B	x_1/x_2	0,186	$\approx 57/70$	inversão

Leitura: $0,129 \ll 0,5$ não é ruído — é **convenção invertida**: com features semânticas o LLM ainda *separa* os grupos ($\approx 0,87$ invertendo os rótulos), mas sem âncora semântica nos nomes o mapeamento rótulo $\leftrightarrow$ classe é arbitrário. O prior atua nos **nomes de classe**, não só nas features.

Fase E (few-shot, teste = 30): o LLM adota a convenção dos *exemplos*: acc vs métrica-alvo $0,744 \rightarrow 0,957$ ($0 \rightarrow 40$ -shot); acc vs real $\approx 0,15$ (a métrica-alvo herdou a convenção invertida da Fase A) — o Perceptron baseline treinado nos mesmos exemplos acompanha ($0,11$ – $0,19$): consistência entre aprendizes, não defeito do LLM.

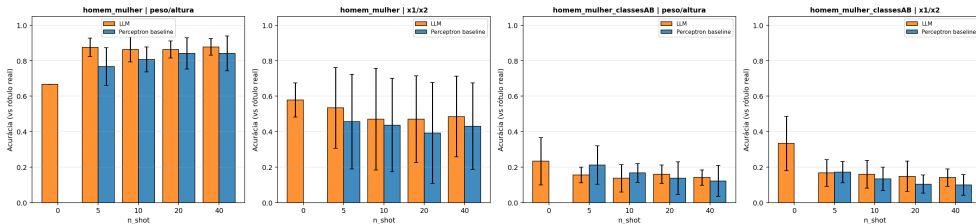
Bloco 3 — Problema G (peso $\times$ altura) — curvas LLM como aprendiz por variante

Fase E (problemas externos) — curva de aprendizado



Bloco 3 — Problema G (peso $\times$ altura) — LLM vs Perceptron baseline

Fase E externa — LLM in-context vs Perceptron treinado nos mesmos exemplos



Bloco 3 — Quando o LLM ganha?

O que está sendo feito: comparação cruzada entre 3 cenários em $n_{\text{shot}} = 10$: (i) peso $\times$ altura com prompt semântico, (ii) peso $\times$ altura com prompt neutro (x_1, x_2), (iii) meia-lua. Mede **acurácia vs ground truth** do LLM contra o baseline Perceptron treinado nos mesmos exemplos. Isola *quando* o LLM tem vantagem sobre um classificador clássico.

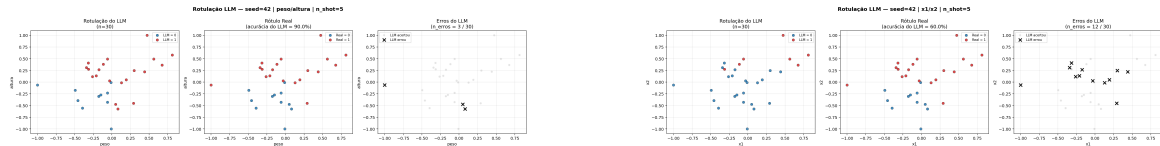
Cenário ($n_{\text{shot}} = 10$)	LLM acc_real	Perceptron baseline real
peso/altura semântico	0,863	0,807
x_1/x_2 neutro	0,470	0,437
meia-lua	não coletada (colapso classe única)	

Interpretação:

- Na base **real**, o LLM supera o Perceptron baseline nos **dois** prompts.
- A **margem** é maior com prior semântico; e só com o prior a acurácia *absoluta* é alta ($\sim 88\%$ vs $\sim 47\%$ no neutro).
- A meia-lua não pôde ser comparada (LLM colapsou no zero-shot).
- Vantagem do LLM vem dos *priors do treinamento*, não do ICL per se.

Bloco 3 — Visualização ponto-a-ponto das rotulações do LLM

Atende adendo do e-mail 22:06.



Esq.: prompt SEMÂNTICO (peso/altura). Dir.: prompt NEUTRO (x_1/x_2). Mesmos pontos, classes muito diferentes.

- **Priors semânticos são reais:** $\text{acc_real peso/altura} \sim 88\%$ vs $x_1/x_2 \sim 47\%$.
- **R4 (ellipse)** recupera a forma teórica mas *overfit-a* com 100 amostras (fid. sobe, acc. real vai ao acaso).
- **LLM vs Perceptron:** na base real o LLM **supera** o Perceptron baseline em todos os n_{shot} (+2,2 a +10,7 p.p.) — resultado *revertido* vs. execuções antigas.
- **Conclusão:** vantagem do LLM vem dos priors semânticos; a meia-lua não pôde ser avaliada (colapso).

Dados em `final_cross_linearity.csv` (mediana 3 seeds, NNLS):

Problema	R2→R4 fid.	R2→R4 acc_real	Esperado
A/B/C lineares	$\sim +2$ p.p. (ruído)	—	sem ganho ✓
Meia-lua (oráculo)	~ 0 (satura 86,7%)	—	teto da diagonal
Peso $\times$ altura (neutro)	+19 p.p.	-16 p.p.	overfit $\times$

Padrão: R3/R4 só compensa com não-linearidade real e dados suficientes; na meia-lua a métrica diagonal satura mesmo ajustada ao rótulo verdadeiro (precisa de termo off-diagonal).

Robustez entre seeds (3 seeds $\times$ 3 reps)

Métrica	Média $\pm$ desvio
Fidelidade Problema A	81,3% $\pm$ 4,4 p.p.
Consistência Problema B	68,7% $\pm$ 5,0 p.p.
Consistência Problema C	87,3% $\pm$ 1,9 p.p.
Kappa Problema B	0,31 $\pm$ 0,12
Kappa Problema C	0,73 $\pm$ 0,04
Cosseno de $\hat{W}$ entre seeds	$> 0,99$

Desvio ≤ 5 p.p. entre seeds — **H5 sustentada**. A direção de $\hat{W}$ é estável (cosseno $> 0,99$); a razão w_1/w_2 é o parâmetro mais ruidoso (CV $\sim 54\%$).

Comparação	Δ observado	Veredito
Zero→10-shot (Problema B, Cons., H2)	+0,255, IC [0,15; 0,33], $d=6,4$	SIG
Easy vs Hard ($n_{\text{shot}} = 5$)	+0,223, IC [0,10; 0,34]	SIG, H4 refutada
Easy vs Hard ($n_{\text{shot}} = 40$)	-0,095 (0,88 vs 0,98)	REVERTIDO
R2 vs R4 (meia-lua, oráculo)	~ 0 (satura 86,7%)	sem ganho
Cosseno W_{perc} , W_{nls}	0,994	robusto

Diagnóstico de γ no Perceptron (item b reunião 30/04, $\sim 520\text{s}$): figura `final_10_gamma_convergence.png` confirma convergência monotônica.

- 1 **Point-biserial** (margem $\times$ acerto, por seed): $r \approx 0,28\text{--}0,39$, $p < 0,001$ — maior margem, mais acertos.
- 2 **Mann-Whitney U** (margem em acertos vs erros): $p < 0,001$ nas 3 seeds — erros concentrados em pontos de baixa margem.
- 3 **Kruskal-Wallis** (ordem de exemplos $\times$ acurácia): $p \geq 0,14$ (n.s.) em $n_{\text{shot}} = 5/10/20$ — ordem não altera significativamente.

Síntese das hipóteses (por bloco)

Bloco	Hipótese	Veredito
I	H1 (consistência)	Parcial: C sim ($\kappa_C=0,73$); B fraco ($\kappa_B=0,31 < 0,5$)
I	H2 (few-shot amplifica)	Sustentada
II	H3 (LLM como aprendiz)	Sustentada (Problema E linear: +40 p.p.)
II	H4 (exemplos difíceis)	REFUTADA em n_{shot} pequeno; <i>revertida</i> em $n_{\text{shot}} = 40$
Trans.	H5 (estabilidade)	Sustentada ($\sigma \leq 5$ p.p.)

- 1 Existe um **critério decisório estável** no LLM (H1) — ortogonal a correção.
- 2 **Few-shot amplifica** a concordância LLM–métrica (H2).
- 3 LLM **aprende critério de perito** via ICL (H3); mas baselines clássicos (LR/SVM/kNN) o *superam* no perito **linear** em todos os n_{shot} .
- 4 **Easy** supera **hard** em poucos exemplos — mas hard reverte em $n_{\text{shot}} = 40$.
- 5 **R3/R4** só ajuda em não-linearidade real E com dados suficientes; na meia-lua a métrica diagonal satura ($\sim 87\%$) mesmo vs verdade; em peso $\times$ altura overfit com 100 amostras.
- 6 Vantagem do LLM em peso $\times$ altura vem dos **priors semânticos** do treinamento.

- Métrica diagonal não captura interações entre features.
- 2D apenas — problemas reais são multidimensionais.
- $\text{Peso} \times \text{altura}$ tem só 100 amostras (pouco para 4 features).
- 1 LLM testado (gpt-4o-mini).
- Sem controle de versão do LLM entre execuções: alias gpt-4o-mini sem *snapshot* datado (coleta 30/06–01/07/2026).
- Flip de $T=0$ medido pela auditoria: 4,753% $\Rightarrow$ teto de $\sim 95\%$ para qualquer métrica de concordância com o LLM.

Obrigado!

Orientador: Prof. Raul Fonseca Neto (UFJF)
Programa de Pós-Graduação em Informática

Perguntas?

Formulação primal (Eq. 28):

$$\max \gamma \quad \text{s.t.} \quad d_W(x_i, c_k) - d_W(x_i, c_\ell) + \lambda \alpha_i \geq \gamma \|w\|, \quad w \geq 0, \quad \alpha \geq 0$$

Regra de correção (Eq. 29):

$$\begin{aligned} w &\leftarrow w(1 - \eta\gamma/\|w\|) - \eta \cdot ((x_i - c_\ell)^2 - (x_i - c_k)^2) \\ \alpha_i &\leftarrow \alpha_i + \eta \end{aligned}$$

Busca binária (Eq. 31): $\gamma^{(t+1)} \leftarrow \gamma^{(t)} + \Delta$. Hiperparâmetros do artigo: $\eta = 0,001$, $C \in [0,1; 1,0]$.

$$\min_w \|Aw - b\|^2 \quad \text{s.t. } w \geq 0, \quad A_{i,j} = (x_{ij} - c_{kj})^2 - (x_{ij} - c_{\ell j})^2, \quad b_i = 1$$

Método de conjunto ativo via `scipy.optimize.nnls`. Tempo polinomial, determinístico, ótimo global do problema convexo. Zero hiperparâmetros.

	Perceptron	NNLS
Método	Gradiente projetado	QP (Lawson-Hanson)
Hiperpar.	η, C, Δ_γ	nenhum
Determinismo	Não	Sim
Ótimo global	Não garantido	Garantido (convexo)

Validação empírica nesta execução: cosseno entre W 's = 0,994.

Caso emblemático (peso $\times$ altura, x_1/x_2 , 4 features, média 3 seeds):

- Fidelidade NNLS $\approx 78\%$ (métrica decora as decisões erradas do LLM).
- Acurácia vs real $\approx 49\%$ (nível do acaso — LLM sem prior semântico).

Diagnóstico: 4 pesos $\times$ 70 amostras de treino = decora ruído.

Mitigações: regularização L1/L2, mais dados, CV k -fold, seleção automática.

- Ahuja & Orlin (2001): formulação clássica em PL.
- Schultz & Joachims (2003): margem SVM-style.
- Coelho, Borges, Fonseca Neto (CILAMCE 2017): perceptron estruturado.

Diferenciação: aplicação a LLMs (interface texto), método black-box, independente de arquitetura.

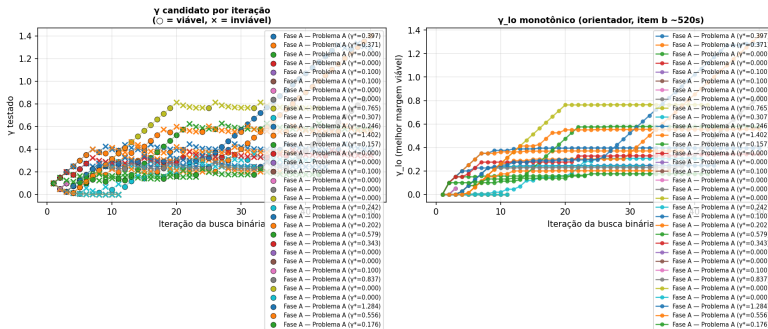
- Brown et al. (2020): GPT-3 e few-shot.
- Min et al. (2022): "no role of ground truth", "label space matters".
- Wei et al. (2022): emergent abilities.
- Lyu et al. (2023): prompts canônicos — alinhado com refutação H4 em few-shot pequeno.

- Xing et al. (2002): original Mahalanobis metric learning.
- Davis et al. (2007): ITML.
- Schultz & Joachims (2003): SVM-style margin.
- Diagonal: Liu et al. (2010), Kedem et al. (2012).

D1 — Reprodução + diagnóstico γ

python src/dissertacao_mestrado.py

Convergência da busca binária em γ — Perceptron Estruturado (CILAMCE 2017, Eq. 31)



Diagnóstico da busca binária em γ (item b reunião 30/04 ~520s): γ_{lo} monotônico crescente confirma convergência.

- LLM evolui silenciosamente entre execuções.
- Temperatura 0 NÃO garante determinismo (batching GPU).
- Usamos $3 \text{ seeds} \times 3 \text{ reps} = 9 \text{ observações/config}$.
- Estocasticidade residual típica $< 2\%$ entre reps da mesma seed.
- Auditoria offline (`src/audit_interactions.py`): flip de $T=0 = 4,753\%$ (1 384 de 29 116 queries repetidas divergiram) $\Rightarrow$ ruído de medição quantificado; fidelidade/ κ devem ser lidos contra um teto de $\sim 95\%$.
- Alias `gpt-4o-mini` sem *snapshot* datado — fixar/registrar o *snapshot* do provedor em execuções futuras.